

What It Is

A point-of-sale system built specifically for a pharmacy. Runs as both a **web app** (browser, port 8082) and a **packaged Windows desktop app** (Electron). No cloud dependency for core operations — all data is stored locally.

Tech Stack

Layer	Technology
Frontend	Pure HTML + vanilla JavaScript — no frameworks, no build tools
Database	SQLite via sql.js (WebAssembly) — runs entirely in the browser
Persistence	IndexedDB — saves the SQLite DB between sessions
Desktop App	Electron — wraps the web app into a .exe installer for Windows
Config Encryption	AES-GCM — sensitive settings (API keys, etc.) encrypted at rest
Payment Terminal	Clover via local bridge server (clover-local-pay/)
Pharmacy Data	WinRx / Pharmacy Dashboard API — patient lookups, Rx data, drug catalog
Product Catalog	McKesson WEBCAT — syncs drug/OTC catalog with item numbers and pricing

Project Structure

```
pharmacy-pos/  
index.html ← Entry point (script tags with ?v=N cache busting)  
css/style.css ← All styles, CSS custom properties for theming  
js/  
db.js ← SQLite schema, all DB read/write functions  
config.js ← AES-GCM encrypted settings (IndexedDB)  
auth.js ← PIN-based staff login  
app.js ← App bootstrap, screen router, theme loader  
screens/
```

```
login.js ← PIN pad + first-run setup
pos.js ← Main POS screen (scan, cart, payment)
patient.js ← Patient profile + AR ledger
transaction.js ← Transaction history + void
settings.js ← All settings tabs + product catalog management
reports.js ← Sales, tax, order suggestions reports
api/
pharmacy-dashboard.js ← WinRx/PD API wrapper
mckesson.js ← WEBCAT catalog sync
clover.js ← Clover payment terminal
email.js ← Email report delivery
utils/
tax.js, print.js, report-print.js, audit.js, scheduler.js
parsers/
barcode.js ← Scan routing (Rx vs OTC vs UPC)
webcat.js ← WEBCAT file parser
electron-app/
main.js ← Electron main process (window, print, SQL server)
preload.js ← Electron <-> renderer bridge (contextBridge)
package.json ← electron-builder config
clover-local-pay/ ← Node.js bridge to Clover terminal (LAN)
pos-worker/ ← Cloudflare Worker (pharmacy dashboard proxy)
setup.bat ← Windows first-time setup
start.bat ← Windows one-click launch (web version)
```

Key Architecture Decisions

1. No Build Tools

Every JS file is a plain `<script>` tag. Adding a feature = create a file, add a script tag, bump `?v=N`. No webpack, no npm for the frontend, no transpilation.

2. Dual Mode: Web vs Electron

Web mode: served by any static file server on port 8082, uses Cloudflare Worker as API proxy to WinRx.

Electron mode: `main.js` spins up an embedded Express server that connects directly to WinRx SQL Server (no internet needed). When `window.__WINRX_LOCAL_API__` is set, ALL API calls bypass Cloudflare and hit local SQL.

3. SQLite in the Browser

`sql.js` (WebAssembly SQLite) runs the entire database in memory. After every write, it exports a binary snapshot to IndexedDB. On load, it restores from IndexedDB. The Electron app uses `loadFile()` so the IndexedDB origin is stable across restarts.

4. Schema Migrations

New columns are added via safe ALTER TABLE ... ADD COLUMN statements wrapped in try/catch — they silently succeed on new DBs and safely no-op on existing ones. No migration versioning needed.

5. Cache Busting

Every script tag uses ?v=N. When a file changes, bump N in index.html. **Always copy index.html along with any changed JS files** — without it the old version is still requested.

Electron App — Windows Deployment

Build Command

```
cd electron-app
npm install
npm run build-windows # produces dist/Pharmacy POS Setup.exe
```

Quick-Copy Updates (no rebuild needed)

For JS/HTML-only changes, copy files directly into the installed app at **C:\PharmacyPOS\resources\app**:

Mac source file	Windows destination
index.html	C:\PharmacyPOS\resources\app\index.html
js/db.js	C:\PharmacyPOS\resources\app\js\db.js
js/screens/*.js	C:\PharmacyPOS\resources\app\js\screens\
js/api/*.js	C:\PharmacyPOS\resources\app\js\api\
js/utils/*.js	C:\PharmacyPOS\resources\app\js\utils\

Files that require a full rebuild (packed inside app.asar):

- electron-app/main.js
- electron-app/preload.js
- electron-app/package.json

Generating a transfer zip (Mac → Windows)

```
cd /path/to/pharmacy-pos
zip -j ~/Desktop/pos-update.zip index.html js/db.js js/screens/reports.js
```

Database Schema — Key Tables

Table	Purpose
staff	Staff PINs, names, roles, email, phone
transactions	Each sale — totals, status, payment method, shift reference
transaction_items	Line items per transaction (Rx, OTC, custom products)
payments	Payment records linked to transactions

products	McKesson catalog synced from WEBCAT (item no, UPC, DIN, price)
custom_products	Manually added products with price override support
patients	Local patient cache synced from WinRx
config	AES-GCM encrypted key/value settings store
audit_log	All events logged with staff PIN + timestamp
cash_movements	Cash in/out records during shifts
shifts	Shift open/close with float and cash reconciliation

WinRx / Pharmacy Dashboard Integration

How it works:

- In Electron mode: main.js connects to **WinRx SQL Server** directly using the mssql npm package
- Endpoints are exposed as local Express routes (e.g. /getRxTx, /getPatient, /getPatients)
- **window.__WINRX_LOCAL_API__** is injected by preload.js pointing to the local server
- PharmacyDashboardAPI checks for this global — if present, all calls go to local SQL; no internet needed
- In web mode: calls route through the Cloudflare Worker at pos-worker/

Patient auto-link on Rx scan:

1. Barcode scanned → parsed as Rx number + branch code by barcode.js
2. getRxTx SQL query runs **two separate queries** — Rx+drug join and patient lookup are independent so one failing cannot block the other
3. _normalizeRxTx() passes patient: patientData in the response
4. POS screen upserts patient to local DB and links to the transaction automatically

McKesson Order Suggestions

How items get McKesson numbers:

WEBCAT catalog sync stores mckesson_item_no in the products table. When generating order suggestions, _findProduct() matches sold items back to the catalog using 4 strategies in order:

1. Exact UPC	Matches upc_unit or gtin_unit exactly
2. Stripped UPC	Removes leading zeros then matches upc_unit / gtin_unit
3. DIN	Matches drug identification number
4. Description	Exact case-insensitive description match (fallback)

Order workflow:

1. Select date range → sold OTC items appear with suggested reorder quantities
2. McKesson # column is **editable inline** — click any field to type a number, it saves automatically on blur
3. Adjust quantities as needed using the Order Qty column
4. Export → McKesson TXT file (format: ITEM_NO TAB QTY, one item per line)
5. Confirm prompt → marks exported items as ordered (mckesson_ordered_at timestamp set in DB)
6. Ordered items are **hidden from suggestions for 30 days** (toggle to show all)

Tech Team Directives

Follow these rules whenever making changes to the Pharmacy POS codebase.

1	Never edit dist/ folder The electron-app/dist/ folder is build output and gets overwritten on every build. Always edit source files in the project root.
2	Always bump ?v=N in index.html When changing any JS file, increment the version number on its script tag in index.html and always ship index.html with the update. Without this, Electron/Chrome loads the old cached version.
3	DB schema changes via migrations only Add new columns only via the migrations array in db.js init() using ALTER TABLE ... ADD COLUMN inside a try/catch block. Never modify CREATE TABLE for existing columns.
4	main.js changes require a full rebuild Any Electron main process logic (SQL connections, printing, window management) lives in electron-app/main.js which is packed into app.asar. Changes require npm run build-windows.
5	Test both Web and Electron modes Changes to pharmacy-dashboard.js affect both modes. Use the _localApi() check to branch behaviour between Cloudflare Worker (web) and local Express/SQL (Electron).
6	Clover bridge startup Started automatically by Electron via spawn() in main.js. For web mode, must be started manually or via start.bat. Configuration lives in clover-local-pay/.env.
7	Electron receipt printing Uses webContents.print() with silent: true. Do NOT set a pageSize override — let the printer driver handle it. Use marginType: 'printableArea'.
8	Config is always encrypted Never store plaintext API keys or credentials in code or files. All credentials go through Config.set(key, value) which AES-GCM encrypts to IndexedDB.

Quick Reference — Common Tasks

Task	How
Add a new screen	Create js/screens/myscreen.js, add script tag with ?v=1 in index.html, register in app.js router
Add a DB column	Add ALTER TABLE ... ADD COLUMN line to the migrations array in db.js init(), bump db.js version

Change a setting key	Use <code>Config.set(key, value)</code> — never write to <code>localStorage</code> or plain files
Update Windows app (JS)	Copy changed files to <code>C:\PharmacyPOS\resources\app\</code> , restart app
Update Windows app (main.js)	Rebuild: <code>cd electron-app && npm run build-windows</code> , reinstall .exe
Transfer files Mac→Win	<code>zip -j ~/Desktop/update.zip index.html js/...</code> then copy zip to Windows
Force reload in Electron	Press <code>Ctrl+Shift+I</code> to open DevTools, then <code>Ctrl+Shift+R</code> for hard reload
Check which JS is loaded	DevTools Console: <code>document.querySelector(script tag).getAttribute('src')</code>